Gestion RH

Application de gestion des ressources humaines, construite bloc par bloc (consignes du cours : photos dans `doc/cours/`). Bloc 1 : demandes de personnel (Manager → HR → Direction → Finance). Bloc 2 en cours : offres d'emploi.

Partie	Techno
`backend/`	Java, Spring Boot, Spring Security (session + cookie), JPA
`frontend/`	Vue 3 (`<script setup>`), Vue Router, Vite
`base/`	PostgreSQL

> **Rappel** : mettre à jour `doc/conception/base/schema.md` (sode adino).

Démarrer

Prérequis : Java + Maven, Node `^22.18` ou `>=24.12`, PostgreSQL ≥ 13 (base `gestion_hr`, `postgres`/`postgres` en local).

```
base/reset.sh                       # recrée la base : init.sql + data.sql
+ demo.sql (--sans-demo, -y)
cd backend && mvn spring-boot:run   # API sur http://localhost:8080 (./mvnw
ne marche pas : .mvn/ manque)
script/run/run-front.sh             # front sur http://localhost:5173, /api
est redirigé vers le backend
script/test/test.sh                 # tests d'API (backend lancé) ;
script/test/test.sh demande pour un seul dossier
cd backend && mvn test              # tests Java (sur la base locale,
annulés à la fin)
```

Comptes de test (`manager`/`manager123`, `hr`/`hr123`…) : voir `doc/user.md`.

Structure

```
doc/
├── cours/                photos des consignes du cours, par bloc (ne pas
modifier)
├── conception/
│   ├── base/schema.md    schéma relationnel : SOURCE DE VÉRITÉ de la base
│   ├── regle-metier/     règles métier décidées, une page par bloc
(bloc1.md…)
│   ├── design/           maquettes
│   ├── exemple-contrats/ modèles de contrats réels (référence pour les
offres)
```

```
│   └── old/                anciens documents, gardés pour l'historique : ne
font plus foi
├── routes/api/             contrat d'API par fonctionnalité (demandes.md…)
├── user.md                 comptes, rôles, authentification
└── Bloc5.md, qol.md        notes de cours / idées pour plus tard
base/
├── init.sql                schéma (suit schema.md)
├── data.sql                données de départ : statuts, comptes de test,
référentiel
├── demo.sql                demandes de démonstration dans tous les statuts
└── reset.sh                remet la base à zéro
backend/src/main/java/gestion/hr/
├── <fonctionnalité>/       auth/, demande/, referentiel/… : une
fonctionnalité = un dossier
│   ├── Controller/  Service/  Repository/  Model/  Dto/  Config/
└── commun/Exception/       ErreurMetier + GestionErreurs (erreurs JSON {
code, message })
frontend/src/
├── api/                    appels au backend (client.js commun, un fichier
par fonctionnalité)
├── auth/                   session de l'utilisateur connecté
├── router/index.js         routes + droits d'accès (lus sur l'utilisateur
renvoyé par le backend)
├── views/<dossier>/<Nom>View.vue        pages (une route = une vue)
├── components/<dossier>/<Nom>.vue       layout/, ui/ (réutilisable
partout), demande/, offre/…
├── assets/css/<dossier>/<Nom>.css       styles d'un composant : même
dossier, même nom
├── assets/css/global/                   couleurs, balises HTML, classes
utilitaires
└── utils/                  fonctions pures partagées (dates, texte,
chargement…)
script/
├── run/                    lancement (run-front.sh)
└── test/<fonctionnalité>/ tests d'API en curl (valeurs-absurdes.sh),
lib.sh commun, test.sh lance tout
```

Normes

1. La documentation passe avant le code

L'ordre est toujours le même :

1. **Règle métier** décidée → écrite dans `doc/conception/regle-metier/blocN.md`.
2. **Base** → `doc/conception/base/schema.md` d'abord, puis `base/init.sql` (et `data.sql` / `demo.sql` si besoin).
3. **API** → `doc/routes/api/<fonctionnalité>.md` (routes, corps JSON, erreurs) : c'est le contrat entre back et front.
4. **Code** backend puis frontend, en suivant ces documents.
5. Si le code a dû s'écarter d'un document, on corrige le document **dans le même commit**.

Un document dans `doc/conception/old/` ne fait plus foi : on ne s'en sert pas comme référence.

Le PDF `doc/cours/Cours_Conception_Logiciel_Gestion_RH.pdf` **n'est pas le cours** : il a été généré par IA et contredit beaucoup de règles métier. On ne s'en sert ni comme référence ni comme source d'idées métier. Les règles viennent des consignes (photos de `doc/cours/`) et des décisions écrites dans `doc/conception/regle-metier/`.

2. Langue et nommage

- Tout est en **français** : noms de classes, méthodes, variables, routes, commentaires, messages d'erreur, commits.
- Java : `PascalCase` pour les classes (`DemandeService`, `CreationDemande`), `camelCase` pour le reste ; sous-dossiers de couche en majuscule (`Controller/`, `Service/`…) comme l'existant.
- Base : tables et colonnes en `camelCase` (`demandePoste`, `idDepartement`), clés étrangères `id<Table>`. Hibernate garde les noms tels quels (`PhysicalNamingStrategyStandardImpl`).
- Routes API : `/api/<fonctionnalité>` en kebab-case (`/api/demandes/mes-demandes`).
- Vue : composants `PascalCase.vue`, pages `<Nom>View.vue`, routes nommées en kebab-case.
- Codes d'erreur en `MAJUSCULES_SOULIGNÉES` (`VALIDATION`, `ACCES_REFUSE`, `TRANSITION_INTERDITE`).

3. Base de données

- `schema.md` et `init.sql` disent toujours la même chose. Chaque table de `init.sql` a un commentaire repris de `schema.md`.
- Les **id des tables de référence** repris par un enum Java (ex. `status` ↔ `Statut.java`) ne changent jamais.
- Les mots de passe sont stockés en **BCrypt** (`doc/user.md` explique comment en générer un).
- Une donnée de démo va dans `demo.sql`, une donnée indispensable au fonctionnement dans `data.sql`.
- Après un changement de schéma : `base/reset.sh`, puis relancer le backend (`ddl-auto=validate` vérifie que les entités suivent).

4. Backend

- **Une fonctionnalité = un dossier**, puis une couche par sous-dossier. On n'importe pas le `Repository` d'une autre fonctionnalité depuis un `Controller`.
- **Controller** : aucune règle, il appelle le service et renvoie un DTO. La Javadoc de la classe liste ses routes.
- **Service** : seul point d'entrée de la fonctionnalité, c'est lui qui ouvre les transactions (`@Transactional`, `readOnly = true` pour les lectures). Quand il grossit, on le découpe en classes **package-private** du même package (comme `CreationDemande`, `DecisionDemande`, `LectureDemande`, `ConversionDemande`) : les appels depuis l'extérieur ne changent pas.
- **Règles et droits côté serveur** : l'utilisateur vient de la session (`UtilisateurCourant`), jamais du JSON. Ce que le front doit savoir (menu, droits) est renvoyé par l'API (`peutCreerDemande`, `aMoiDeDecider`…).
- **Erreurs** : on lève `ErreurMetier` (`validation`, `accesRefuse`, `introuvable`…) ; `GestionErreurs` la transforme en `{ code, message }`. Le message est lisible par l'utilisateur final, en français.

- **Une seule source** pour les listes fixes : un enum (`Role`, `Statut`) porte codes, libellés et règles. Pas de copie ailleurs.
- On n'expose jamais une entité JPA : on passe par un DTO (`Dto/…Request`, `Dto/…Response`). Le mot de passe ne sort jamais.
- Injection par constructeur, champs `private final`.
- Chaque classe commence par un commentaire qui dit **à quoi elle sert** et ce qu'il faut savoir pour la modifier.

5. Frontend

- **Pages** dans `views/`, **morceaux** dans `components/` ; un composant réutilisable dans plusieurs fonctionnalités va dans `components/ui/` (`Tableau`, `Erreur`, `RechercheSelect`, `Progression`). Avant d'en écrire un, regarder s'il existe déjà, et le généraliser plutôt que le copier.
- **Appels HTTP** uniquement dans `api/`, via `appel()` de `api/client.js` (cookies + CSRF + `ErreurApi`). Une vue n'appelle jamais `fetch` directement.
- **Rien de métier en dur dans les vues** : rôles, droits, libellés, statuts, étapes viennent du backend. Les données encore statiques (backend pas prêt) restent dans `api/<fonctionnalité>.js`, avec un commentaire « Données statiques » à l'endroit où elles sont utilisées, et disparaissent dès que la route existe.
- **CSS** : pas de `<style>` dans les `.vue`. Le style d'un composant va dans `assets/css/<dossier>/<Nom>.css`, importé dans `assets/main.css` (dans l'ordre du général au précis). Couleurs et rayons : variables de `css/global/variables.css` (`--color-primary`, `--radius`…), pas de couleur en dur.
- Erreurs affichées avec le composant `Erreur`, chargements avec `useChargement` (`utils/chargement.js`).
- Ergonomie voulue : la page est remplie (pas de grand vide), un champ pas encore utilisable est **grisé** plutôt que caché, les tableaux trient au clic sur l'en-tête et défilent dans le composant.
- Formatage : `npm run format` (Prettier : sans point-virgule, guillemets simples, 100 colonnes) avant de committer.

6. Tests

- Toute route nouvelle ou modifiée a :
  - un test Java dans `backend/src/test/java/gestion/hr/<fonctionnalité>/` (`@Transactional` : la base n'est pas modifiée) ;
  - des cas « valeurs absurdes » dans `script/test/<fonctionnalité>/valeurs-absurdes.sh` (négatif, 0, date passée, id inconnu, champ vide, rôle interdit, sans CSRF…). Un script qui crée des données les supprime à la fin.
- Avant une PR : `mvn test` et `script/test/test.sh` passent, et la fonctionnalité a été essayée dans le navigateur avec **chaque rôle concerné**.

7. Git

- `main` reçoit uniquement des **Pull Requests**. Une branche par bloc : `bloc-1`, `bloc-2`… ; `release/bloc-N` quand un bloc est livré.
- Un commit = un changement compréhensible seul. Message en français qui dit **ce qui change** (« Filtre par département sur le formulaire d'offre » plutôt que « refacto » ou « modif »).
- Code, doc et SQL d'un même changement vont dans le même commit.

- Jamais de `target/`, `node_modules/`, `dist/`, fichiers de cookies (`jar`) ou fichiers de l'IDE personnels.

Travailler avec l'IA (Claude Code)

Ce qui suit vient des conversations déjà menées sur ce projet : ce qui a bien marché, et ce qui a fait perdre du temps.

Bien formuler la demande

- **Dire où regarder** : « dans frontend », « va dans doc », « en te référençant à `schema.md` », « au contrat d'API ». Ouvrir le fichier concerné dans VS Code : l'IA voit quel fichier est ouvert.
- **Une demande = un sujet.** Les longues demandes « fais A, puis B, puis C » ont souvent été interrompues et reprises. Mieux vaut enchaîner des demandes courtes.
- **Dire ce qu'il ne faut pas toucher** : « sans toucher au composant du tableau », « ne touche à rien d'autre ».
- **Insister sur les détails importants** (« ce détail est important ») et les écrire ensuite dans `doc/conception/regle-metier/`. L'IA garde une mémoire personnelle, mais elle n'est vue que par la personne qui l'utilise : **la doc du dépôt est la seule mémoire partagée de l'équipe**.
- **Réutiliser l'existant** : « en utilisant les composants déjà faits », « prends exemple sur la page détail ».
- Pour une règle métier floue, lui demander d'abord de **poser des questions** ou d'expliquer (« comment identifie-t-on les supérieurs ? ») avant de coder. La décision prise va dans la doc.
- Les skills installés aident : `coding-propre` (todo list, changements ciblés), `debug-methodique` (comprendre un bug sans corriger), `documentation`, `ux-navigation-review`.

Garder le contrôle

- **Interrompre tôt** (Échap) dès que l'IA part dans la mauvaise direction, et corriger en une phrase (« Non, ce sont des étapes statiques, elles doivent toutes être supprimées »).
- **Relire chaque modification** (`git diff`) avant de committer. Demander à la fin : « écris-moi un rapport de ce que tu as fait et modifié par rapport à mes demandes » pour repérer les oublis.
- **Vérifier soi-même** : lancer les tests et ouvrir la page avec les bons comptes. « Ça compile » ne veut pas dire « c'est juste ».
- **Nouvelle conversation par tâche** (ou par bloc) : une conversation très longue mélange les sujets et l'IA se trompe davantage. Pour reprendre, lui donner la doc à lire plutôt que de rejouer l'historique.
- Les pages ou données temporaires faites pour « voir à quoi ça ressemble » sont marquées comme telles (première ligne du fichier) et supprimées avant la PR.
- L'IA commit ou pousse **seulement si on le lui demande**, et jamais directement sur `main`.

Données

- Ne jamais coller de vraies données de salariés (noms, CIN, salaires, contrats signés) dans une conversation : utiliser les données de `demo.sql`.

À ne pas faire

Base

- × Modifier `init.sql` sans `schema.md` (ou l'inverse).
- × Changer les id de `status` ou d'une table reprise par un enum Java.
- × Passer `spring.jpa.hibernate.ddl-auto` à `update` ou `create` : le schéma vient de `base/init.sql`, point.
- × Mettre un mot de passe en clair dans la base, ou un vrai mot de passe dans `data.sql`.

Backend

- × Mettre une règle métier ou un contrôle de droit dans un `Controller` ou dans le front seulement.
- × Lire le demandeur, le valideur ou le rôle dans le JSON envoyé par le front.
- × Renvoyer une entité JPA au lieu d'un DTO.
- × Lever une exception brute (`RuntimeException`, `IllegalArgumentException`) pour une règle métier au lieu de `ErreurMetier`.
- × Dupliquer codes ou libellés d'un enum (`Role`, `Statut`) ailleurs que dans l'enum.
- × Utiliser `./mvnw` (cassé) : utiliser `mvn`.

Frontend

- × Écrire en dur dans une vue les rôles, droits, statuts, libellés ou étapes du workflow.
- × Appeler `fetch` ailleurs que dans `api/client.js`.
- × Ajouter un `<style>` dans un `.vue`, ou une couleur en dur au lieu d'une variable CSS.
- × Copier-coller un composant pour l'adapter : le généraliser (`components/ui/`).
- × Cacher un champ pas encore disponible : le griser.
- × Laisser des pages d'exemple, des données de test ou des `console.log` dans une PR.

Git et doc

- × Pousser directement sur `main`.
- × Committer `target/`, `node_modules/`, `dist/` (c'est déjà arrivé avec `backend/target`).
- × Messages de commit vides de sens (« refacto », « modif », « truc »).
- × Changer une route ou un format JSON sans mettre à jour `doc/routes/api/`.
- × Prendre `doc/conception/old/` comme référence.
- × Se fier au PDF `Cours_Conception_Logiciel_Gestion_RH.pdf` (généré par IA, règles métier fausses), ni le donner à lire à l'IA.

IA

- × Accepter du code sans l'avoir relu ni testé.
- × Ajouter au dépôt un document généré par IA (cours, règles, contrat) sans l'avoir vérifié : il sera pris pour une source fiable.
- × Demander « fais tout le bloc » en une seule fois.
- × Laisser une décision importante seulement dans la conversation ou la mémoire de l'IA : elle va dans `doc/`.
- × Coller des données personnelles réelles dans une conversation.